Week 5

Task 1

Maximum Subarray

Aim:-To find the maximum possible sum of a subarray in a circular array, considering both normal and wrap-around subarrays.

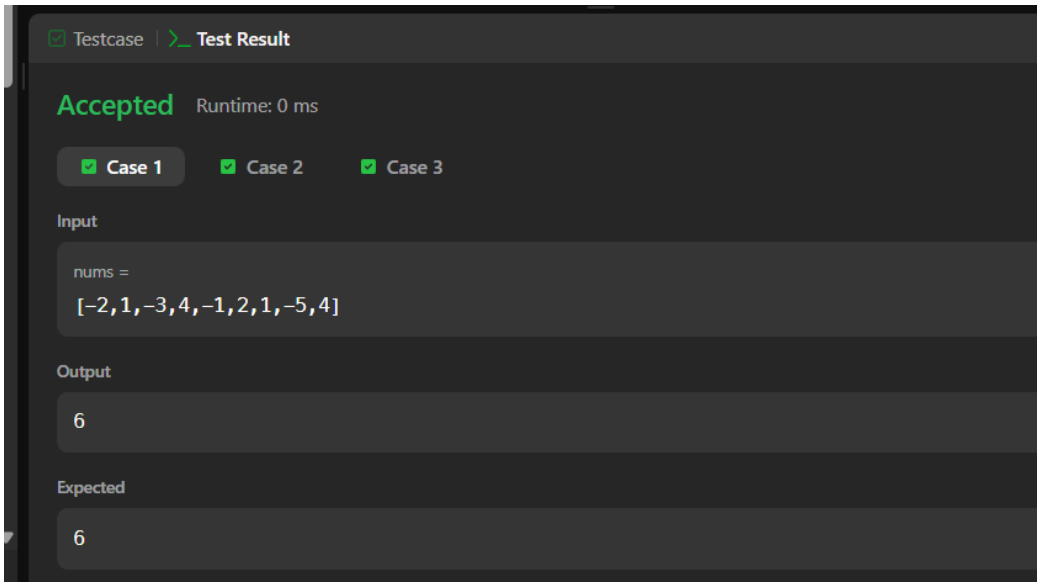
Algorithm:-

1. Use Kadane's algorithm to find the maximum subarray sum.
2. Use Kadane's algorithm to find the minimum subarray sum.
3. Compute the total sum of the array.
4. Calculate circular sum as totalSum − minSubarraySum.
5. If all elements are negative, return the normal maximum subarray sum.
6. Otherwise, return the maximum of normal sum and circular sum.

Program:-

```
class Solution {
   public int maxSubArray(int[] nums) {
      int max = nums[0], cur = nums[0];
      for (int i = 1; i < nums.length; i++) {
         cur = Math.max(nums[i], cur + nums[i]);
         max = Math.max(max, cur);
      }
      return max;
   }
}
```

Output:-



Result:- Thus,To find the maximum possible sum of a subarray in a circular array, considering both normal and wrap-around subarrays has successfully executed.

Task 2

Subarray Division

Aim:- To find the number of contiguous segments of length m in the chocolate bar such that the sum of elements equals d**.**

Algorithm:-

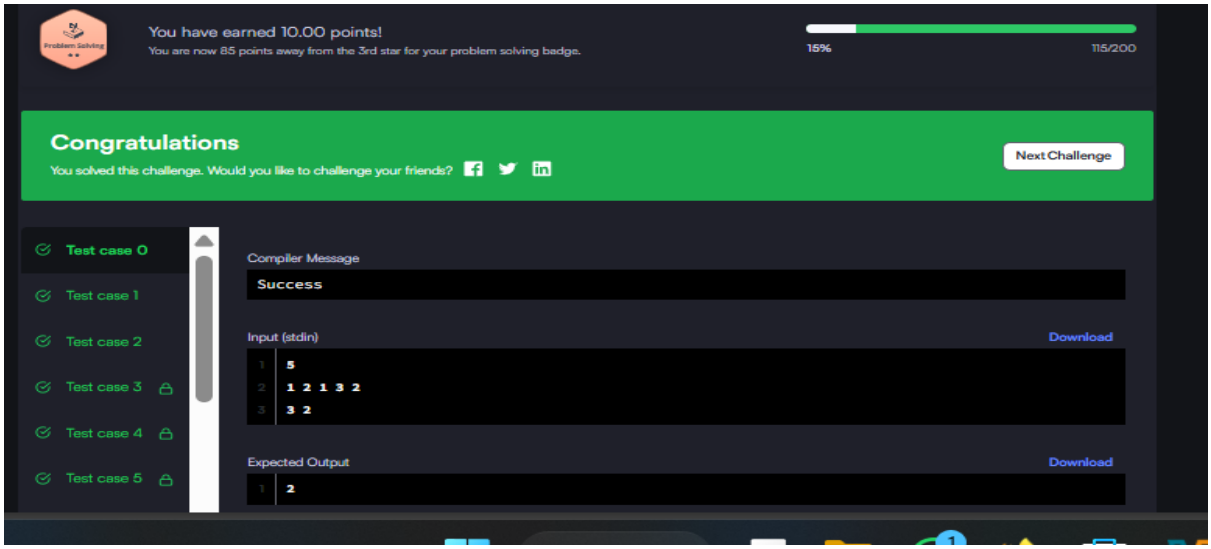
1. Initialize count = 0 and sum = 0.
2. Traverse the list from index 0 to n-1.
3. Add the current element to sum.
4. If window size exceeds m, subtract the element leaving the window (i - m).
5. When window size becomes m, check:
    a. If sum == d, increment count.
6. After traversal, return count

Program:-

```
import java.util.*;

class Result {

   public static int birthday(List<Integer> s, int d, int m) {

      int count = 0;

      for (int i = 0; i <= s.size() - m; i++) {

         int sum = 0;

         for (int j = 0; j < m; j++) {

            sum += s.get(i + j);

         }

         if (sum == d) count++;

      }

      return count;

   }

}
```

```
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        List<Integer> s = new ArrayList<>();
        for (int i = 0; i < n; i++) s.add(sc.nextInt());
        int d = sc.nextInt();
        int m = sc.nextInt();
        System.out.println(Result.birthday(s, d, m));
    }
}
```

Output:-



Result:- Thus, To find the number of contiguous segments of length m in the chocolate bar such that the sum of elements equals d has successfully executed .

Task 3

The Maximum Subarray

Aim:- To compute the maximum sum of a contiguous subarray and the maximum sum of a non-contiguous subsequence from a given integer array.

Algorithm:-

1. Initialize currentSum and maxSubarray with the first element.
2. Traverse the array:
3. Update currentSum = max(arr[i], currentSum + arr[i]).
4. Update maxSubarray with the maximum value.
5. For subsequence:
6. Add all positive elements to maxSubsequence.
7. Track the maximum element in the array.
8. If no positive elements exist, set subsequence sum to the maximum element.
9. Return both sums.

Program:-

```
import java.util.*;

class Result {

    public static List<Integer> maxSubarray(List<Integer> arr) {

        int maxSubarray = arr.get(0);

        int currentSum = arr.get(0);

        int maxSubsequence = arr.get(0);

        int subsequenceSum = arr.get(0) > 0 ? arr.get(0) : 0;

        for (int i = 1; i < arr.size(); i++) {

            int val = arr.get(i);

            currentSum = Math.max(val, currentSum + val);

            maxSubarray = Math.max(maxSubarray, currentSum);

            if (val > 0) subsequenceSum += val;

            maxSubsequence = Math.max(maxSubsequence, val);

        }
```

```
        if (subsequenceSum > 0) maxSubsequence = subsequenceSum;
        return Arrays.asList(maxSubarray, maxSubsequence);
    }
}
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int t = sc.nextInt();
        while (t-- > 0) {
            int n = sc.nextInt();
            List<Integer> arr = new ArrayList<>();
            for (int i = 0; i < n; i++) arr.add(sc.nextInt());
            List<Integer> res = Result.maxSubarray(arr);
            System.out.println(res.get(0) + " " + res.get(1));
        }
    }
}
```

Output:-

Result:-Thus,To compute the maximum sum of a contiguous subarray and the maximum sum of a non-contiguous subsequence from a given integer array was successfully executed.

Task 4

Maximum Sum Circular Subarray

Aim:- To find the maximum sum of a contiguous subarray from the given integer array.

Algorithm:-

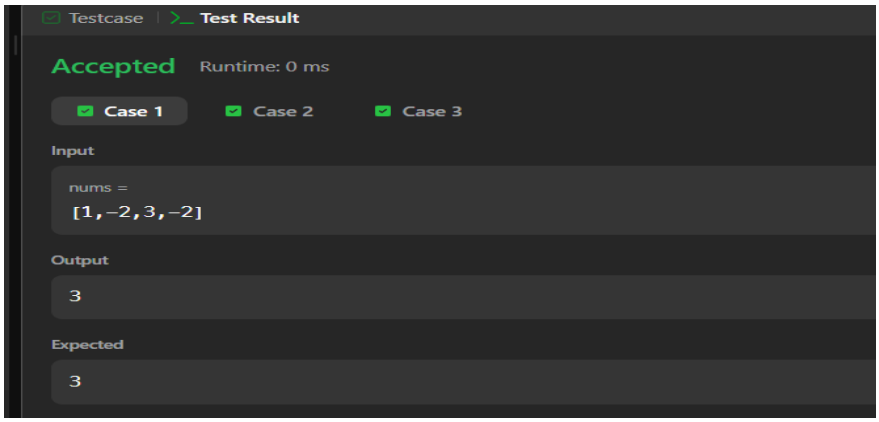
1. Initialize currentSum and maxSum with the first element.
2. Traverse the array from the second element.
3. For each element, update currentSum as the maximum of the current element or the sum of current element with previous sum.
4. Update maxSum if currentSum is greater.
5. Return maxSum.

Program:-

```
class Solution {

   public int maxSubarraySumCircular(int[] nums) {

      int total = 0;

      int maxSum = nums[0], curMax = nums[0];

      int minSum = nums[0], curMin = nums[0];

      for (int i = 1; i < nums.length; i++) {

         int val = nums[i];

         curMax = Math.max(val, curMax + val);

         maxSum = Math.max(maxSum, curMax);

         curMin = Math.min(val, curMin + val);

         minSum = Math.min(minSum, curMin);

         total += val;

      }

      total += nums[0];

      if (maxSum < 0) return maxSum;

      return Math.max(maxSum, total - minSum);
```

```
    }
}
```

Output:-



Result:-Thus, To find the maximum sum of a contiguous subarray from the given integer array has successfully executed.